LES ÉLÉMENTS DE BASE

Les Fonctions

Visibilité des variables

Les variables déclarées dans une fonction ne seront utilisables que dans celle-ci.

Les variables déclarées dans le script ne seront pas accessibles dans une fonction.

Les deux espaces sont complètements indépendants.

Portée des variables

- variable locale
- variable globale
- variable statique

Nota : pour accéder dans une fonction à une variable globale on utilise le mot clé **global**. Une variable statique est une variable locale persistante. On la crée à l'intérieur de la fonction avec le mot clé **static**.

LES ÉLÉMENTS DE BASE

Les Fonctions

Exemple

```
$message = 'Nombre de passage : ';

function compteur($mode='') {
        global $message;
        static $nb=0;
        $nb++;
        // On incrémente le nombre de camions
        if($mode == 'affiche') {
        echo $message . $nb;

       }

}
```

Nota : Sauf précision, tout paramètre passé à une fonction, est passée par valeur (donc par copie). La seule exception à cette règle concerne les objets, qui sont toujours passés par référence depuis PHP 5.

GÉNÉRALITÉS : INCLUSION DE FICHIERS

- **include** et **require**
  - Permettent d'inclure le contenu du fichier spécifié en argument
  - Différentes au niveau de la gestion des erreurs
  - **include** peut-être conditionnel alors que **require** ne peut l'être.

- **include_once** et **require_once** => éviter des inclusions multiples.

include émettra un message d'**alerte** (warning) si elle ne trouve pas un fichier spécifié tandis que **require** émettra une erreur fatale dans le même cas. Autrement, **require** est bloquant tandis que **include** ne l'est pas.

GÉNÉRALITÉS : LES COOKIES

- **Cookies**

- Fichiers texte courts stockés par le navigateur sur l'ordinateur de l'utilisateur, à la demande du serveur web.
- Suites d'informations transmises en en-tête HTTP par le serveur au client.
- Informations retournée par le navigateur (client) lors de chaque interrogation du même serveur HTTP.

- **Catégories** : **sans durée d'expiration** (fichier en mémoire vive)**, avec durée de vie** (fichier sur disque)

- **Ecriture et modification** : fonction **setcookie** dont la syntaxe est la suivante :

  bool **setcookie** (string $name [, string $value [, int $expire = *0* [, string $path [, string $domain [, bool $secure = *false* [, bool $httponly = *false*]]]]]])

- **Lecture** : variable superglogale $_COOKIE

GÉNÉRALITÉS : LES COOKIES

- **Cookies**

- Suppression : Envoie d'un cookie du même nom mais sans valeur.

  setcookie("nom_cookie");

  setcookie ("TestCookie", "", time() - 3600); // précaution sur date de validité

  unset($_COOKIE["nom_cookie"]); // précaution sur l'objet

- Limitations et sécurité

  20 cookies/domaine

  4Ko par cookie

  autres filtres niveau navigateur

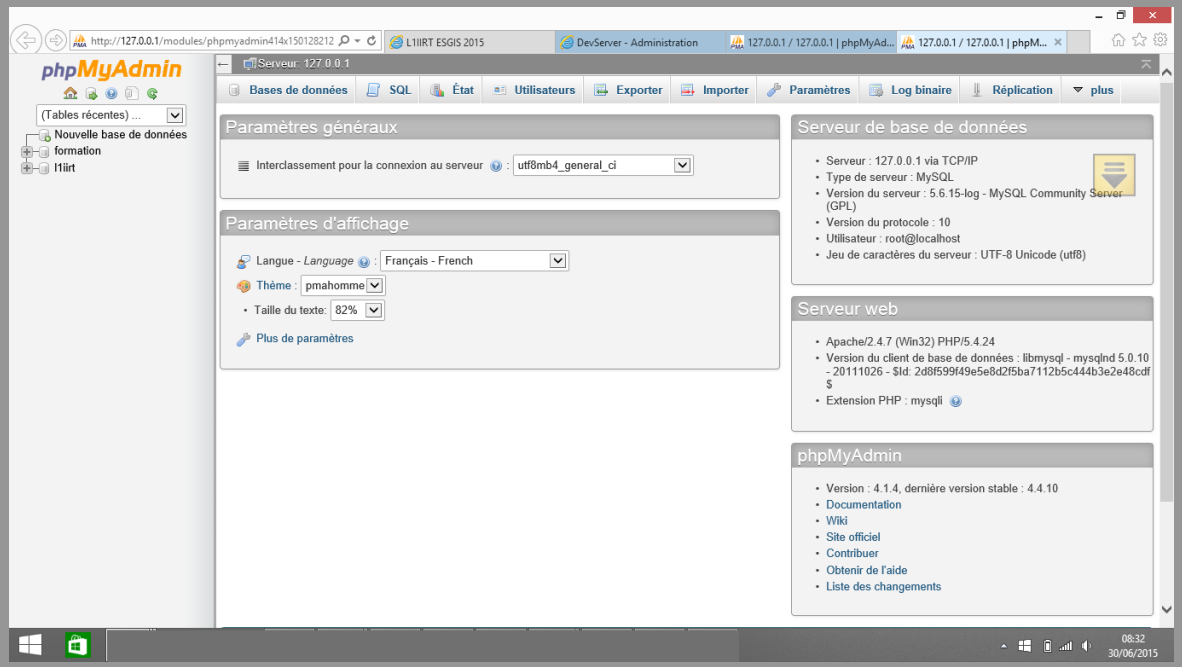
  aucune sécurité (falsification, etc.)

- TP : cas pratique, ***Outil de personnalisation d'affichage***

GÉNÉRALITÉ: PHPMYADMIN

phpMyAdmin est une interface (application) Web développée en PHP et distribuée sous licence GNU GPL pour administrer les bases de données MySQL.

Son interface se présente comme suit :

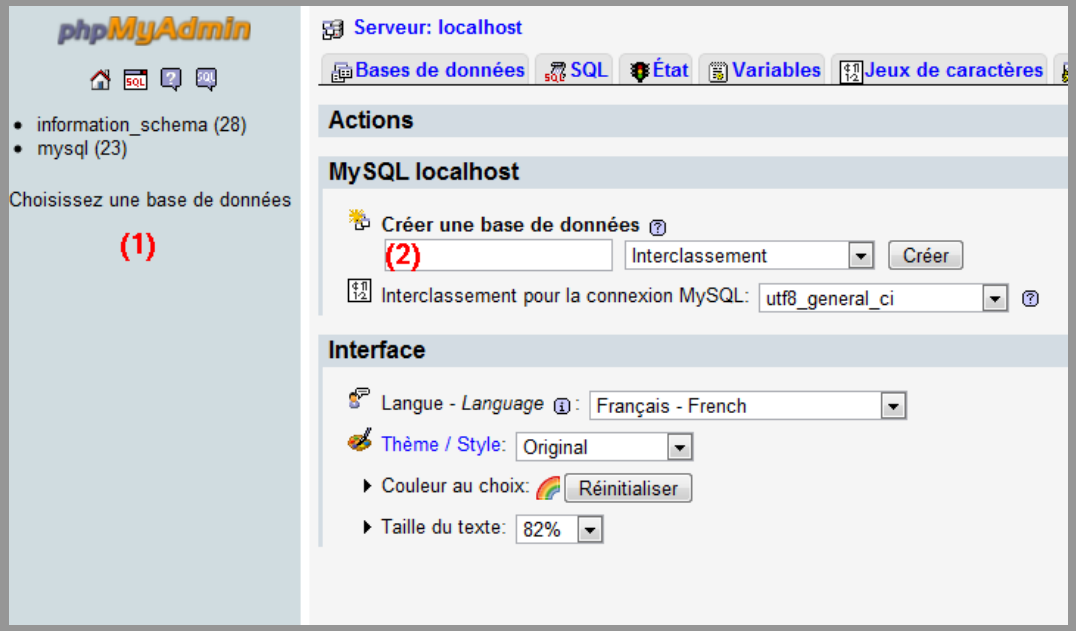


Elle est accessible depuis un navigateur. Mais le chemin d'accès est fonction de l'installation du serveur et|ou de l'hébergement. Il existe souvent des raccourcis via un menu au niveau de la barre des tâches pour la lancer.

GÉNÉRALITÉ: PHPMYADMIN

Notre objectif n'est pas de présenter en détail phpMyAdmin, mais d'en citer quelques fonctionnalités de base.

- **Créer une base de données** : en sélectionnant l'onglet **« Bases de données »**

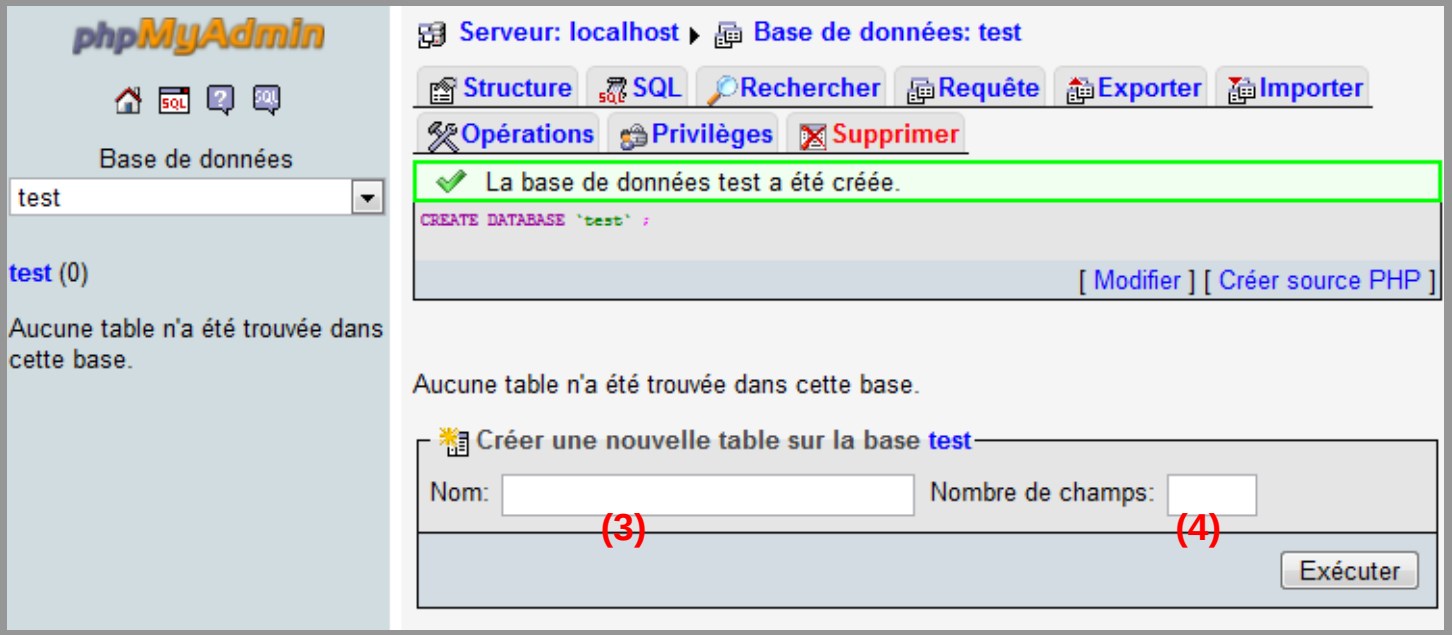


Une fois le nom de la base de données saisi dans la zone (2), on pourra cliquer sur le bouton créer pour procéder à la création.

GÉNÉRALITÉ: PHPMYADMIN

- **Créer une table**

A la fin de la création d'une nouvelle base de données, il est automatiquement proposé dans l'interface qui s'affiche, de créer une table. On peut aussi sélectionner, dans la partie gauche, une base de données pour y créer une nouvelle table.

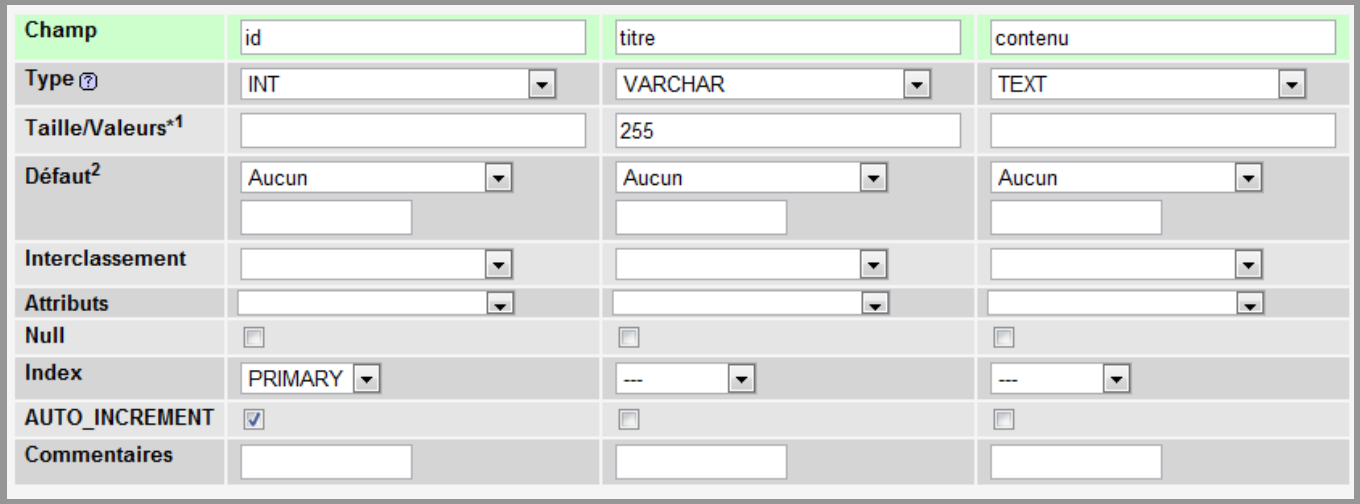


Pour créer une table, il suffit de saisir son nom dans la zone (3) et le nombre de ses champs dans la zone (4) puis cliquer sur le bouton **« Executer »**. Une nouvelle interface va s'afficher pour indiquer la structure de la table (nom des champs ainsi que le type et le format des données qu'ils peuvent contenir.

GÉNÉRALITÉ: PHPMYADMIN

- **Créer une table (suite)**

La première ligne de la figure ci-dessous permet de nommer les différents champs de votre table. Il faut noter que cette ligne peut aussi se présenter sous forme de colonne (verticalement) si la table a beaucoup de champs.



La seconde ligne permet de choisir le type de la donnée contenue dans chaque champ de la table; etc.

Dans l'exemple ci-dessus nous pouvons distinguer les champs « id », « titre », et « contenu » qui sont de type respectifs « INT », « VARCHAR », « TEXT »

Index : active l'indexation du champ. Ici, il s'agit d'une clé primaire « PRIMARY », donc identifiant unique.

AUTO_INCREMENT : permet au champ de s'incrémenter tout seul à chaque ajout d'enregistrement.

A la fin des configurations, il faudra cliquer sur le bouton **« sauvegarder »** pour enfin créer la table. Ce bouton n'est pas visible sur la figure ci-dessus mais il est souvent juste en bas des champs et des paramètres.

GÉNÉRALITÉ: PHPMYADMIN

- **Actions possibles sur une table**

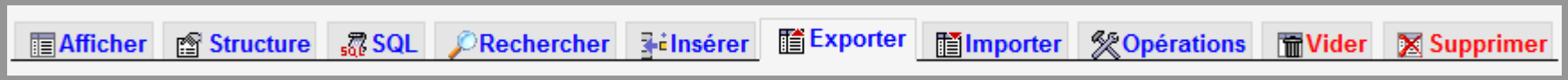
Une fois créée, les actions de bases que l'on peut effectuer sur une table sont les suivantes :

- Insérer une entrée
- Supprimer une ou des entrées
- Modifier une entrée
- Afficher les entrées
- Vider une table
- Supprimer la table
- Effectuer toutes sortes de requêtes autorisées sur la table

- **Actions utiles sur une base de données**

- Sauvegarder (Exporter) une base de données MySQL
- Restaurer (Importer) une base de données MySQL
- Renommer
- Supprimer

Ces différentes actions sont disponibles depuis l'interface graphique de phpMyAdmin suivant que vous sélectionnez à gauche, une base de données ou une table. Il suffit de chercher un tout petit peu en explorant les onglets



PHP ET BASES DE DONNÉES MYSQL

La plupart des opérations réalisées dans l'interface de phpMyAdmin sont directement réalisable en écrivant du code php et en faisant des requêtes. Pour nous simplifier la tâche, nous allons supposer l'existence de bases de données et de tables. Dons, pas de code pour ces deux choses. phpMyAdmin s'en chargera pour nous.

- **Accès à une base de données MySQL depuis PHP**

L'approche recommandée aujourd'hui pour accéder à une base de données reste PDO mais comme à ce niveau de formation, aucune notion d'approche objet n'est encore enseignée, nous allons, pour des raisons pédagogiques, présenter une approche dépréciée mais marche.

Cette approche utilise des fonctions PHP conçues pour attaquer exclusivement des bases de données MySQL.

L'accès à une base de données MySQL se fera généralement en 3 étapes. Une quatrième étape peut cependant exister suivant le type de requête exécutée.

- **Etape 1 : Connexion au serveur hébergeant la base de données**
- **Etape 2 : Sélection de la base de données sur laquelle on veut travailler**
- **Etape 3 : Exécution de la requête voulue**
- **Etape 4 : Exploitation, au besoin, des résultats de la requête**

Chacune de ses étapes est réalisée avec une ou des fonctions PHP dédiées à MyQSL.

Dans le cadre de ce cours nous allons utiliser des fonctions préfixées par **mysqli_**

PHP ET BASES DE DONNÉES MYSQL

- **Etapes 1 et 2 : Connexion au serveur hébergeant la base de données et sélection de cette dernière**

  Il est question d'ouvrir ou de réutiliser une connexion à un serveur MySQL et de sélectionner base de donnée sur ce serveur.

  On va utiliser la fonction **mysqli_connect** dont la syntaxe complète est la suivante :

mysqli mysqli_connect ([string $host = ini_get("mysqli.default_host") [, string $username = ini_get("mysqli.default_user") [, string $passwd = ini_get("mysqli.default_pw") [, string $dbname = "" [, int $port = ini_get("mysqli.default_port") [, string $socket = ini_get("mysqli.default_socket")]]]]])

On y retrouve plusieurs paramètres optionnels dont les plus importants sont :

- Le nom du serveur
- Le nom de l'utilisateur qui se connecte
- Son mot de passe
- Le nom de la base de données

En cas de succès, la fonction retourne **l'identifiant de connexion** MySQL. Dans le cas contraire, elle retourne **FALSE**.

PHP ET BASES DE DONNÉES MYSQL

Exemple de code PHP pour les étapes 1 &2 :

$link = mysqli_connect('localhost','root','', 'nom_bdd') or die ('Impossible de se connecter au serveur');

- **Etape 3 : Exécution de la requête voulue**

  On utilise le fonction **mysqli_query** dont la syntaxe complète est la suivante :

  mixed mysqli_query (mysqli $link , string $query [, int $resultmode = MYSQLI_STORE_RESULT])

On y retrouve trois paramètres dont 1 est optionnel :

- $query : conbtient la requête sous forme de chaine de caractères
- $link : représente la ressource (identificateur) caractérisant la connexion MySQL retourné par la fonction mysqli_connect
- $resultmode : Elle a pour valeur soit la constante MYSQLI_USE_RESULT, soit la constante MYSQLI_STORE_RESULT, suivant le comportement désiré. Par défaut, la constante MYSQLI_STORE_RESULT est utilisé..

La valeur retournée par la fonction mysqli_query est fonction du type de requête exécutée.

Pour les requêtes du type SELECT, SHOW, DESCRIBE, EXPLAIN et les autres requêtes retournant un jeu de résultats, la fonction retourne une **ressource en cas de succès**, et **FALSE en cas d'erreur**.

Pour les autres types de requêtes, INSERT, UPDATE, DELETE, DROP, etc., la fonction retourne **TRUE en cas de succès** et **FALSE en cas d'erreur**.

Exemple de code PHP :

$result = mysqli_query('SELECT * FROM NOM_DE_VOTRE_TABLE');

PHP ET BASES DE DONNÉES MYSQL

- **Etape 4 : Exploitation, au besoin, des résultats de la requête**

  Cette étape n'a de sans que pour les requête de type SELECT.

  On utilise les fonctions préfixées par **mysql_fetch_** et éventuellement une boucle pour parcourir les enregistrements retournés.

mysqli_fetch_xxx (où xxx représente « array », « row », « assoc », « object ») retourne un tableau ou un objet qui contient la ligne demandée dans le résultat **$result** et déplace le pointeur de données interne d'un cran ou pas.

Exemples de code PHP :

```
//pour récupérer le résultat courant
$lign=mysqli_fetch_assoc($result);

//Ou encore pour boucler sur tous les résultats
while($lign=lign=mysqli_fetch_assoc($result) {
   // traiter $lign
}
```

Nota : La page qui suit présente des exemples complets d'utilisation.

PHP ET BASES DE DONNÉES MYSQL

Dans un premier temps on va y utiliser le fichier nommé **connect.inc.php** dont le code est le suivant :

```
<?php
        $host="localhost";
        $login="root";
        $passw="";
        $bdd="votre_bdd";
                $link=mysqli_connect($host,$login,$passw, $bdd) or die("Erreur de connexion au serveur");
?>
```

- **Exemple de sélection d'enregistrement d'une table**

```
<?php
        require_once('connect.inc.php');
        $req="select * from nom_votre_table";
        $result=mysqli_query($link, $req);
        while($lign=mysqli_fetch_assoc($result)) {
                echo $lign["champ1"] ." ". $lign["champ2"] ." ". $lign["champ3"] ... . ."<br/>";
        }
?>
```

- **Exemple d'insertion d'un enregistrement dans une table**

```
<?php
        require_once('connect.inc.php');
        $req="insert into nom_votre_table value('valeur1', 'valeur2',..., 'valeurN')";
        $result=mysqli_query($link, $req);
?>
```

PHP ET BASES DE DONNÉES MYSQL

- **Exemple de sélection d'enregistrement d'une table**

```
<?php
include_once('connect.inc.php');
$req="select * from nom_votre_table";
$result=mysqli_query($link, $req);
while($lign=mysqli_fetch_assoc($result))
{
    echo $lign["champ1"] ." ". $lign["champ2"] ." ". $lign["champ3"] ... . ."<br/>";
}
?>
```

- **Exemple d'insertion d'un enregistrement dans une table**

```
<?php
        include_once('connect.inc.php');
        $req="insert into nom_votre_table value('valeur1', 'valeur2',..., 'valeurN')";
        $result=mysqli_query($link, $req);

?>
```

PHP ET BASES DE DONNÉES MYSQL

Quelques fonctions mysqli

mysqli_fetch_row — Retourne une ligne de résultat MySQL sous la forme d'un tableau
mysqli_fetch_array — Retourne une ligne de résultat MySQL sous la forme d'un tableau associatif, d'un tableau indexé, ou les deux
mysqli_fetch_assoc — Lit une ligne de résultat MySQL dans un tableau associatif
mysqli_num_rows — Retourne le nombre de lignes d'un résultat MySQL
mysqli_affected_rows — Retourne le nombre de lignes affectées lors de la dernière opération MySQL

Et beaucoup d'autres ci-dessous dont vous pouvez vérifier la syntaxe d'utilisation sur le site **www.php.net**

mysqli_close
mysql_data_seek
mysqli_errno
mysql_error
mysqli_connect_error
mysqli_escape_string
mysqli_real_escape_string

mysqli_free_result
mysqli_num_rows
mysqli_get_client_info
mysqli_info
mysqli_commit
mysqli_prepare
mysqli_ping

GÉNÉRALITÉS : LES SESSIONS

- **Sessions**

  Réponse aux limitations des cookies => **Stockage coté serveur** des données sensibles des utilisateurs.

  Solution => Sessions

  Objectif => préservations données sensibles contre vol, falsification, etc.

  Principe => - Attribution d'un identifiant de session,
  - Envoie de l'identifiant de session au client,
  - Enregistrement coté serveur (fichier) des informations de session,
  - Enregistrement de l'identifiant au niveau du client (cookie)
  - A chaque requête des pages appropriées du serveur, le navigateur renvoie l'identifiant
  - Le serveur s'en sert pour récupérer dans le fichier correspondant, les informations préalablement stockées

GÉNÉRALITÉS : LES SESSIONS

- **Sessions**

  Initialisation => fonction **session_start(),** placée en haut du script, avant toute sortie vers la page web; comme pour les header (en-tête)

  Si identifiant (provenant du client) alors lecture données stockées sur serveur

  Sinon, création nouveau identifiant, transmission au client (pour stockage), création fichier correspondant (coté serveur)

  Lecture, Ecriture => variable superglobale **$_SESSION[]**

```
// Initialisation de la session
   session_start() ;

// Écriture d'une variable de session
   $_SESSION["nom_variable"] = 'valeur' ;
```

Suppression d'une session => session_destroy() , après initialisation

session_destroy() => suppression du fichier de session (coté serveur) mais pas variables de session.

```
unset($_SESSION);
```

Nota : Des exemples de codes se trouvent sur les deux pages suivantes

GÉNÉRALITÉS : LES SESSIONS

```
<?php   //Code appelé par le formulaire d'authentification
                session_start();
if(isset($_SESSION["login"]))  //si une session existait, on la détruit
{
                $_SESSION = array();
                session_destroy();
}
if(isset($_POST["login"]) && isset($_POST["passwd"]) && ! (empty($_POST["login"]) || empty($_POST["login"])))
// si on vient de soumettre un formulaire d'authentification avec tous les champs bien renseignés
{
    include_once("connexion.inc.php"); // on inclut le fichier contenant les paramètres de connexion
                                                    // à la base de données
    $requet=sprintf("SELECT * FROM votre_table_user WHERE login='%s' AND passw='%s'",
                                                mysql_real_escape_string($_POST["login"]),
                                                md5(mysql_real_escape_string($_POST["passwd"])));
   $result=mysqli_query($link,$requet);
   if(mysqli_num_rows($result)==1)  // on doit trouver un seul enregistrement
   {
       $connecte=mysqli_fetch_assoc($result);  //on récupère le seul résultat dans un tableau associatif
                //on enregistre les paramètres de session qu'on souhaite avoir à disposition. Un exemple ci-dessous
                $_SESSION["login"]=$connecte["login"];
                $_SESSION["nom"]=$connecte["prenom"] . '  ' .$connecte["nom"];
                $_SESSION["profil"]=$connecte["profil"];   //peut être utile si on veut gérer les droits des utilisateurs
                                                                //pour décider de quoi afficher
```

GÉNÉRALITÉS : LES SESSIONS

```
                //redirection vers la page souhaitée

                header("Location:votre_page_membre.php");  // c'est dans cette page que vous allez gérer au besoin
                                                                                // les droits des utilisateurs mais ce n'est pas une
                                                                        // obligation. Tout dépend de ce que vous voulez faire
                }
                else  // En cas de mauvaise identification
                {
                                //redirection vers la page d'authentification
                                header("Location:page_authent.php");
                }
}
else  // En cas de non authentification (non soumission de formulaire d'authentification)
{
                header("Location:page_authent.php");
}

?>
```

GÉNÉRALITÉS : LES SESSIONS

```php
<?php
        //Code appelé pour fermer ou terminer une session
        session_start();
        if(isset($_SESSION["login"]))  //si une session existait, on la détruit
        {
                $_SESSION = array();
                unset($_SESSION);
                session_destroy();
        }
?>
```

GÉNÉRALITÉS : LES SESSIONS

```
<?php
		//Code appelé pour fermer ou terminer une session
		session_start();
	if(isset($_SESSION["login"]))  //si une session existait, on la détruit
	{
		$_SESSION = array();
		unset($_SESSION);
		session_destroy();
	}
?>

<?php
		//Code pour protéger une page par une session
		//(elle ne sera accessible qu'aux personnes ayant ouvert une session)
		session_start();
if(isset($_SESSION["login"]))  //si une session existe
{
		?>
		<!- Mettre tout le code (HTML et autres) de votre page ici -->
		<?php
}
else
header('Location:page_authent.php');
?>
```

Deuxième partie

PHP ET SÉCURITÉ

Internet a envahi notre vie quotidienne de façon incroyable => presque une dépendance à tous égards .

Aujourd'hui tout y passe : informations technologiques, réservation d'une salle de cours, gestion en ligne des informations financières et commerciales, achat , paiement de facture, contrôle à distances d'installations domestiques ou autres.

Problème : Internet est un réseau et tout le monde y a accès => problèmes de Sécurité

La confiance réciproque des parties ne suffit pas => nécessité de protection et de contrôle à tous les niveaux.

Qualité d'une application => Pas seulement une application qui marche.

Il faut qu'elle respecte le cahier des charges et qu'elle fasse **strictement** ce pour quoi elle a été conçu sans laisser la porte ouverte à d'autres **excès , écarts, violations** ou **indiscrétions**.

PHP ET SÉCURITÉ

La sécurité de l'information doit répondre aux besoins fondamentaux suivants :

la disponibilité : assurer l'accessibilité à tout moment des données et services aux ayant droit

l'intégrité : assurer la non altération (accidentelle ou frauduleuse) des données

la confidentialité : rendre non accessibles et/ou non compréhensibles les données et ressources aux personnes non autorisées.

l'authentification : ne donner accès aux ressources qu'aux personnes autorisées en les authentifiant pour s'assurer de leur identité via un processus d'identification.

la non répudiation : assurer l'impossibilité de nier une transaction.

PHP ET SÉCURITÉ

Pour préserver les applications et répondre aux besoins fondamentaux de sécurité, il faut prévenir les applications de certains risques.

Les risques à prévenir

- abus de ressources
- destruction de données
- divulgation de données confidentielles
- détournement de site
- usurpation d'identité
- atteinte à l'image de marque

$$Risque = \frac{(Menaces)\ X\ (Vulnérabilités)}{(Contre-mesures)}$$

La sécurité est tellement complexe que pour la garantir ou s'en approcher il faut l'appréhender à **plusieurs niveaux**. En effet, votre application aura beau être sécurisée, si le serveur web qui l'héberge est mal configuré du point de vue sécurité, alors tous vos efforts pourront être vains. De même si le systèmes d'exploitation (ou sa version) qui héberge le serveur Web présente des failles critiques de sécurité exploitables, tout est également mis en cause.

Plusieurs niveaux et plusieurs acteurs rentrent en jeu dans la mise en œuvre et la garantie de la sécurité.

Nous allons présenter dans ce document un seul niveau de garantie de la sécurité :

- sécurisation de l'application

PHP ET SÉCURITÉ

Sécurité de l'application

Pour être efficace, la sécurité de l'application doit être mise en œuvre à tous les niveaux du cycle de l'application :

Conception

Faire preuve de bon sens dans ses choix

Eviter de faire de la sécurité par l'obscurité votre arme principale. L'obscurité peut ralentir mais n'empêche pas d'avancer.

Opter pour la simplicité tout en garantissant l'efficacité. La simplicité garantie la clarté et la maitrise. La complexité est souvent source de confusion et de problèmes de vulnérabilité.

Bien organiser les fonctionnalités pour les satisfaire sans mettre en péril l'architecture

Ne jamais faire confiance à personne et ne rien négliger . Prévoir des mécanismes pour tout contrôler et tout valider. C'est de la défense en profondeur.

Toujours prévoir un plan de secours pour limiter les dégâts en cas de problèmes

Développement

La plus grande partie du travail se fait à ce niveau

Filtrer toutes **les données entrantes** pour qu'elles ne perturbent pas l'exécution normale des opérations. La méfiance est de règle. Donc il faut tout vérifier à la loupe avant de valider.

Opter pour les solutions qui séparent le traitement (préparation) de la donnée de son intégration (exécution).

PHP ET SÉCURITÉ

Trouver un moyen efficace pour la gestion des jeux de caractères (UTF, ISO, etc.) pour éviter les problèmes d'incompatibilité et laisser une porte ouverte aux injections.

Se protéger contre les différentes sortes d'injections (HTML, SQL, XSS)

Suivre les données à tout moment et s'assurer que les données des variables que vous utilisez sont validées par un mécanisme sécuritaire que vous auriez mis en place. Ainsi cela évitera de courir le risque d'utiliser des données brutes, source d'éventuels problèmes.

Une approche serait qu'après filtrage des données entrantes (par des fonctions, des bibliothèques externes prévues à cet effet) , de placer les données validées dans un autre tableau ou variable avec un nom explicite ($_PROPRE, $_CLEAN, $_VALIDE, etc.)

Veiller à la protection des données sortantes (générées ou validées par PHP) avant de les transmettre à une autres technologies qui va les utiliser (HTML, SQL, XML, LDAP, Shell, etc.)

PHP ET SÉCURITÉ

au besoin faire des tests unitaires

Toujours auditer le code à la fin (soit par des outils, soit par quelqu'un d'autre de confiance) car l'ignorance ou l'inattention peut vous faire glisser dans votre code des sources de vulnérabilité.

Mise en production

A ce niveau, la sécurité se conjugue au quotidien. Il faudra donc surveiller l'application à tout moment pour s'assurer de sa stabilité et de sa qualité.

modérer le contenu (surtout s'il est interactif) pour éviter les écarts et certains propos.

Pour la surveillance, l'analyse et la traçabilité, il faut mettre en place les fichiers logs et les étudier régulièrement.

PHP ET SÉCURITÉ

Maintenance

La maintenance d'une application peut avoir plusieurs sources de motivation.

- Extension
- Correction de bugs
- Changement d'approche de programmation ou de technologie

Dans tous les cas, il s'agit de coder à nouveau. il faut veiller à respecter les recommandations des différentes étapes rentrant dans le cycle de développement d'une application.

Suivre l'actualité et tenir compte des nouvelles recommandations et des correctifs pour faire évoluer l'application pour la sécuriser.

En un mot, restez dans la dynamique de la veille sécuritaire

PHP ET SÉCURITÉ

Quelques exemples d'injection SQL

Pour se connecter frauduleusement

... OR true

... OR 1=1

... OR 'a'='a'

Pour contourner un bannissement

... AND false

... AND 1=2

... AND 'a'>'b'

Pour connaitre le nombre de champs d'une table

... UNION SELECT 1

... UNION SELECT 1,1

... UNION SELECT 1,1,1

Pour retrouver les noms des champs d'une table

... ORDER BY nom_du_champ

... GROUP BY nom_du_champ

Pour retrouver le nom d'une table

... UNION SELECT 1,2,3 FROM nom_de_la_table

PHP ET SÉCURITÉ

Quelques fonction de filtrage ou de protection de données

addslashes : Ajoute des antislashs dans une chaîne
stripcslashes : Décode une chaîne encodée avec addcslashes

htmlentities : Convertit tous les caractères éligibles en entités HTML
html_entity_decode : Convertit toutes les entités HTML en caractères normaux

htmlspecialchars : Convertit les caractères spéciaux en entités HTML
htmlspecialchars_decode : Convertit les entités HTML spéciales en caractères

strip_tags : Supprime les balises HTML et PHP d'une chaîne

urlencode : Encode une chaîne en URL
urldecode : Décode une chaîne encodée URL

mysqli_real_escape_string : Protège une commande SQL de la présence de caractères spéciaux

Quelques fonctions de cryptage ou de hachage

md5 : Calcule le md5 d'une chaîne
sha1 : Calcule le sha1 d'une chaîne de caractères
crypt : Hachage à sens unique (indéchiffrable)
hash : Génère une valeur de hachage (empreinte numérique)

PHP ET SÉCURITÉ

Exemple d'utilisation de fonction de filtrage ou de protection de données

Comme vous l'auriez constaté dans le code exemple de gestion des session, il est recommandé d'utiliser la fonction mysqli_real_escape_string pour protéger les données contre les injections SQL.

Ainsi, au lieu d'une requête :

```
$req="SELECT * FROM votre_table_user WHERE login='" .$_POST['login']  .' and passw='"
                                        .$_POST['passwd'] ."'";
```

On écrira par exemple :

```
$req=sprintf("SELECT * FROM votre_table_user WHERE login='%s' AND passw='%s'",
                                        mysqli_real_escape_string($_POST["login"]),
                                        md5(mysqli_real_escape_string($_POST["passwd"])));
```

sprintf est une fonction PHP qui permet de retourner une chaine formatée au lieu de l'afficher comme **printf** et **echo** le font.

md5 est une fonction de PHP qui retourne le md5 de la chaîne, sous la forme d'un nombre hexadécimal de 32 caractères. C'est une fonction de hachage cryptographique, donc normalement non réversible.

Parmi les fonctions de hachage disponibles dans la plupart des langages de programmation, l'algorithme SHA-1, est à ce jour l'un des plus efficaces. En PHP, la fonction associée à cet algorithme est **sha1**.

Requête préparée : Une autre façon de protéger les requêtes contre les attaques est d'utiliser les requêtes préparées. Une requête préparée, c'est en fait un modèle de requête que l'on enregistre et auquel on donne un nom. On va ensuite pouvoir l'exécuter en l'appelant grâce à son nom, et en lui passant éventuellement un ou plusieurs paramètres.

Sécurité des Sessions

Sécurité :

Cacher les sessions pour lutter contre les vols de session

Eviter tant que possible de définir soi même les identifiants de session (utiliser les identifiants par défaut)

Vérifier l'identifiant de l'utilisateur (stockez le plus d'information sur lui mais jamais son mot de passe)

Etre vigilant et ne jamais faire confiance

PHP ET SÉCURITÉ

Quelques autres conseils

Sécuriser les cookies

Sécuriser les sessions

Faire de l'audit de sécurité de votre code

Pour aller plus loin

PDO
HTTPS
SSL

Nota : La sécurité n'est jamais totale. Restez toujours sur vos gardes.